

beamfs v3:
An EM-Resilient Linux Filesystem
under Read-Path Fault Injection,
with an Audit of the Injector
Technical Report, Version 3

Aurélien Desbrières, independent researcher
ORCID: 0009-0002-0912-9487 aurelien@hackers.camp

Abstract—beamfs is a Linux filesystem that stores every data block as a capsule of sixteen interleaved Reed-Solomon RS(255,239) codewords, decodes on every read, and refuses a read it cannot vouch for. This report measures beamfs 0.1.26 (commit 1bf151d) on Linux 7.3-rc5. Of the 734 tests of the xfstests auto group, all 122 that ran on x86-64 passed, and 122 of the 123 that ran on aarch64; xfstests declined the others, most of them for features beamfs does not implement, and these are counted apart, not as passes. Under single-bit upsets injected into read bios by the emufi injector, on a four-node aarch64 cluster and on USB flash media, beamfs never returned wrong data: it returned the correct file in every exercised case but one, absorbing up to 265 flips during one read of a 256 KiB file, and in that one refused the read, after a flip in an indirect pointer. In the same runs ext4 and ext3 returned silently corrupted data after 1 and 2 to 3 flips respectively, and btrfs refused the read. We audit the injector itself. On this kernel emufi 0.8.1 logs one of its two read hooks one bio before the block it flips; reconstructing the hook of every flip explains all 60 corrections the kernel reported and bounds the load at 2 erroneous symbols per codeword and per decode, against eight correctable. The campaign therefore exercises detection and correction end to end, not the correction limit. Redundancy (btrfs DUP data, ZFS with two copies) also returns correct data, for a full copy; beamfs does it on one medium for 7 % of capacity, while its indirect pointers, in the default configuration, are less protected than ext4’s checksummed extent blocks. We set out what beamfs brings, what it costs, and what v4 takes on. Scope, after the project’s normative statement: beamfs is designed against Family A perturbations (stochastic electromagnetic single-bit and multi-bit upsets, aging silicon bit-rot, environmental radiation, rowhammer-class single-cell flips); Family B perturbations (adversarial bursts exceeding 256 octets per sub-block, IEMI at scale, voltage-glitch attacks at scale) are not claimed. This report measures a subset of Family A. Every number in it is generated from signed run records and cross-checked before typesetting.

Index Terms—Electromagnetic resilience, filesystem, Reed-Solomon, Linux kernel, fault injection, emufi, xfstests, single-event upset, reproducibility

I. INTRODUCTION

A filesystem trusts the bytes a read brings back. Between the medium and the page cache those bytes cross a controller, a bus, DMA buffers and caches, any of which can flip a bit: soft errors in commercial silicon are a measured fact [1], and they matter most where hardware is commercial and the environment is not. Memon et al. [2], [3]

irradiated commercial systems on chip running Linux with 20 to 58 MeV protons and found that, on an i.MX 8M Plus running ext4 on eMMC, filesystem faults and storage-driver errors accounted for 90 % of the observed failures, up to journal aborts and read-only remounts [3]. Conventional filesystems either do not check file data (ext4, ext3) or check it and, holding a single copy, refuse the read (btrfs); they return the data the application wrote only from a second copy of it [4], [5].

beamfs [6] is a Linux filesystem built to return it, or to refuse. Every data block is a *capsule* of sixteen interleaved RS(255,239) codewords [7], [8]; every read decodes it; a block beyond correction fails the read with an error instead of handing back wrong bytes. beamfs descends from FTRFS [9] through an RFC posted in April 2026 [10] and two earlier reports [11], [12]. Section II explains why it is a new filesystem rather than a revision of FTRFS.

What this report adds:

- 1) A measured beamfs 0.1.26: of the 734 tests of the xfstests auto group, all 122 that ran on x86-64 passed, and 122 of the 123 that ran on aarch64; the tests xfstests declined, mostly for features beamfs does not implement, are counted and classified, not passed (Section VII-A).
- 2) A comparison with ext4, ext3, btrfs and squashfs under emulated single-bit upsets on the read path, replicated over three runs, and a four-node cluster campaign (Sections VII-B and VII-C).
- 3) An audit of the injector. emufi 0.8.1 [13] logs one of its two read hooks one bio early on this kernel. We locate the cause in the kernel source, reconstruct the hook of every flip, and check the reconstruction against the kernel’s own correction log (Section VI).
- 4) A fail-closed event on an indirect block, explained from the code, and the residual it exposes (Section VII-D).
- 5) A parallel with the proton campaign of Memon et al., including what that work reports about the hardware it irradiated (Section VIII).
- 6) A positioning of beamfs against redundancy, with its costs and where it is weaker today (Section IX), and the work the next version takes on (Section XI).
- 7) A reproducibility record in which every number of the text is generated from GPG-signed run manifests and frozen by hash (the appendix).

What changed since v2: The v2 report [12] is superseded on five points. (i) Data blocks are now interleaved

capsules (Section III-A); v2 described the alternating layout. (ii) v2 stated that a corrected block is written back to the medium on read; in 0.1.26 the read path corrects into a private copy and does not write back, on-disk repair being left to the background scrubber (Section III-B). (iii) The module is no longer 2198 lines: it is 17348 lines of C and headers at 1bf151d, beyond the 5000-line audit target v2 quoted. (iv) Injection moved from RadFI [14] to emufi. (v) Validation is the xfstests `auto` group, test by test, on two architectures.

II. FROM FTRFS TO BEAMFS

FTRFS [9] proposed a filesystem for the on-board memory of spacecraft: checksums, forward error correction and memory protection over a small radiation-exposed store. Its Linux realisation, posted as an RFC in April 2026 [10] and described in the FTRFS v1 report [11], inherited the concept and its limits. Each limit below is conceptual, not a missing feature: it follows from the threat model or from the place error correction holds in the design, which is why beamfs is a new filesystem with its own on-disk format rather than a further FTRFS revision. Table I sets them side by side.

Threat model: FTRFS assumes radiation-induced upsets arriving as independent events at a low constant rate, the Poisson regime of space radiation. It says nothing about correlated events, bursts, or a deliberate attacker. The soundness theorem of the first beamfs report was stated under that model; fault injection falsified it, and v2 withdrew it [12]. beamfs states two families of perturbation, claims one, and says which (Section IV).

Where correction sits: In FTRFS, Reed-Solomon protects metadata and a CRC guards each block; the decoder runs once a CRC has failed, and the RFC shipped the encoder only [10]. File data, the part an application reads back, had no correction at all, and a CRC that is itself hit can mask or invent a failure. In beamfs every data block is encoded, and the decoder runs on every read whatever any checksum says (Section III-B).

Scale: The FTRFS allocation bitmap is one RS-protected block: 16 codewords of 239 bytes, 30592 bits, so at most 30592 blocks (about 119 MiB at 4 KiB). The beamfs superblock records up to 65535 bitmap blocks.

Kernel integration: The RFC had no `address_space_operations`, and its review recalled the strong aversion to merging new filesystems built on buffer heads [10]. beamfs reads and writes file data through `iomap` (`iomap_read_folio`, `iomap_readahead`, `iomap_file_buffered_write`), but metadata (indirect blocks, bitmap, scrubber) still goes through buffer heads. That objection is narrowed, not lifted.

Evidence: The RFC was tested by mounting a volume and listing an empty directory. beamfs is tested with the xfstests `auto` group on two architectures and under fault injection (Section VII).

TABLE I
FTRFS AND BEAMFS ON THE POINTS THAT SEPARATE THEM.

	FTRFS (2015 concept, 2026 RFC)	beamfs 0.1.26
threat model	radiation only, independent upsets	two families; B explicitly not claimed
file data	CRC, no correction	16 RS codewords per block
decoder	after a CRC failure; encoder only in the RFC	on every read
bursts	not addressed	data interleaved: a burst must exceed 128 bytes; parity grouped: 9 bytes suffice (Section III-A)
bitmap	one block, 30592 blocks max	up to 65535 bitmap blocks
VFS	no aops; buffer heads	iomap for data; buffer heads for metadata
evidence	mount and empty listing	xfstests, two architectures; fault injection

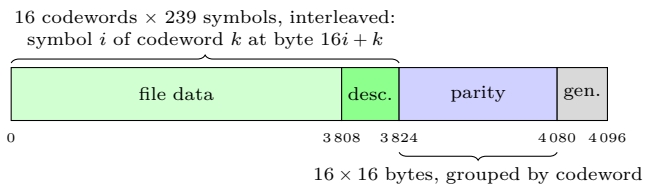


Fig. 1. A beamfs data block under `INCOMPAT_RS_INTERLEAVE`, the layout of every measured volume; byte offsets below, not to scale. The descriptor (bytes 3808 to 3823, checksum and self-identifier) is inside the code and off on the measured volumes; bytes 4080 to 4095 hold the generation number, outside the code.

III. DESIGN OF BEAMFS 0.1.26

A. The capsule

A beamfs data block is 4096 bytes (Figure 1). Its first 3824 bytes are the data symbols of sixteen RS(255,239) codewords over $GF(2^8)$, interleaved: symbol i of codeword k sits at byte $16i + k$. Bytes 0 to 3807 hold file data; bytes 3808 to 3823 hold a checksum and a block self-identifier when the volume is formatted with the optional `DATA_CSUM` and `DATA_SELFID` features, and are otherwise unused. The sixteen parity symbols of codeword k follow, grouped, at byte $3824 + 16k$. The last sixteen bytes carry a generation number outside every codeword. A file therefore takes one block per 3808 bytes: a 256 KiB file takes 69 data blocks.

Each codeword corrects up to $t = 8$ erroneous symbols, so a block survives up to 128 erroneous bytes if they are spread. Interleaving decides what spread means. A burst of up to 128 consecutive bytes in the 3824 coded bytes puts at most eight symbols in any codeword; 129 put nine in one, which is lost. The parity is not interleaved: nine consecutive bytes inside the 16 parity bytes of one codeword are enough to lose it. That region is 256 bytes, 6.25% of the block. No result in this report depends on burst tolerance: the injected upsets are single bits (Section V).

B. Read path

Reading a file range, beamfs reads each data block the page overlaps into a private scratch page, decodes the sixteen codewords there, and copies the requested bytes out. A 4 KiB page straddles two 3808-byte payloads, so a sequential read fetches most blocks twice, each time from the medium. The decoder runs on every read, whatever any checksum says. If any codeword is beyond correction the read fails with `-EIO` and the event is journalled: beamfs refuses rather than returns bytes it cannot vouch for.

Correction is applied to the copy. In 0.1.26 the read path does not write the corrected block back. The on-read repair v2 described was disabled when decoding moved into a private copy; the source states that re-enabling it needs exclusive access to the block for the whole read-modify-write. A background scrubber decodes allocated data blocks at a bounded pace (one block every 100 ms by default) and writes a corrected block back only if its owner and its content have not changed in between and the volume is writable. Errors on the medium are therefore corrected on every read and repaired by the scrubber, not by the read.

C. Metadata and indirect blocks

Inodes carry their own RS parity and a CRC over the whole inode; directory blocks are RS-encoded; the feature bits of every measured volume read `incompat 0x7a100` (per-inode RS, indirect parity, directory RS, full inode CRC, parity-region FEC, interleave) and `ro_compat 0`, which confirms that `DATA_CSUM` and `DATA_SELFID` were off.

An indirect block holds 512 raw 64-bit pointers and has no room for its own code. Its parity lives in a separate region, one slot per indirect block, by default in Reed-Solomon mode. The slot covers the first 3824 bytes, pointers 0 to 477; pointers 478 to 511 are covered by nothing [6] (`known-limitations.md`, 3.38). In Reed-Solomon mode the verify step decodes a *copy* of the indirect block, journals what it found and leaves the buffer alone: an earlier version corrected the live block against stale parity and erased valid pointers, so a verify no longer writes. A correctable error is thus journalled and the pointer is read as it stands. Every pointer is then checked against the data area and the allocation bitmap, and a pointer outside either fails the lookup with `-EUCLEAN`. A flipped pointer that stays inside the data area and lands on an allocated block passes every check; without `DATA_SELFID` nothing detects it. The project recorded one such case on a direct pointer, 15245 wrong bits returned with no kernel signal (comment in `file_inline.c` at 1bf151d). Section VII-D reports what the campaign did to indirect blocks.

IV. SCOPE: DESIGNED AND MEASURED

The project’s data-protection design requires every public artefact to state its scope in the following normative words, reproduced verbatim (`Documentation/data-`

`protection-design.md`, section 7, at 1bf151d; v5.0 names the on-disk format):

beamfs v5.0 protects against Family A perturbations (stochastic electromagnetic single-bit and multi-bit upsets, aging silicon bit-rot, environmental radiation, rowhammer-class single-cell flips). Family B perturbations (adversarial bursts exceeding 256 octets per sub-block, IEMI at scale, voltage-glitch attacks at scale) are not claimed by v5.0. The on-disk format reserves the feature flag bit `BEAMFS_FEATURE_INCOMPAT_SHADOW_PARITY` for a future v5.x revision that adds out-of-band parity with stride placement, targeting Family B coverage. Coexistence between the two schemes is supported by design.

That statement is the design scope. The measured scope of this report is narrower, and every result below is to be read within it:

- *one mechanism*: single-bit flips in the data of read bios, after the device returned them. The medium is not altered: all 867 flips recorded on beamfs are on read bios, and after every attack, injector off, the twelve files of every filesystem read back intact (Section VII-B);
- *one load*: at most two flipped bits per read bio, hence at most two erroneous symbols per codeword per decode (Section VI); multi-bit upsets, bursts and the correction limit of eight symbols are not exercised;
- *one configuration*: on-disk format v5 with interleaving, without `DATA_CSUM` or `DATA_SELFID`, indirect parity in Reed-Solomon mode;
- *one kind of target*: the blocks of one 256 KiB file (its data blocks, plus its indirect block in runs 2 and 3) and, through the injector defect of Section VI, the block that follows each of them.

Nothing here is a physical irradiation, and nothing here measures Family B.

V. EXPERIMENTAL SETUP

Software under test: beamfs 0.1.26, commit 1bf151d, built into one Yocto image per architecture (layer `yocto-beamfs` at 9d43172) with Linux 7.3-rc5 (72d3fcf8) [15], the version both build configurations pin. beamfs is built into the kernel on both (`CONFIG_BEAMFS_FS=y`); the module file the images also carry is never loaded. The two kernels differ in one beamfs option and in their preemption model. The x86-64 kernel enables `CONFIG_BEAMFS_DEBUG_TREE`, which records every pointer stored in an indirect block, checks each later store to the same slot against the record, decodes a copy of every parity-region block it has just encoded, and warns on a violation; these are diagnostics that change nothing on disk, but cost time and can fail a test through the warning. The aarch64 kernel does not enable it. The images, the kernels and the module sources are identified by SHA-256 (the appendix): every source file of the module has the same hash in the commit, in the layer, in the run manifests and in the 6 source trees the two builds compiled. The aarch64 image, 1d60a1be893fa8eae92463082181d77be207393a3a7e18ffff7cc60183963e62, is checked on every node before each bench run. The

guests boot from beamfs: their root filesystem is a beamfs volume.

Machines: One host (spartan, Gentoo Linux, Intel Core i7-12700F) runs four aarch64 guests emulated by QEMU without hardware acceleration (machine virt, Cortex-A57, four vCPUs and 2 GiB each: `beamfs-master`, `compute01` to `compute03`) and one x86-64 guest under KVM (four vCPUs, 8 GiB). In the cluster campaign each aarch64 node attacks a beamfs volume on its own virtual disk. The filesystem comparison runs on `compute01`, each filesystem on its own USB flash drive, five drives on the host, each passed to the guest as a virtio block device.

Workload: For each filesystem and each probability, the bench formats the device (for beamfs, `mkfs.beamfs -s inline -O per_inode_rs`: feature bits `incompat 0x7a100`), writes three directories of three 256 KiB random files with a SHA-256 list per directory (twelve files), and records the extents of the target file `dir-B/file-B2.bin`. An attack reads the target once with the injector off, as a reference; arms the injector; syncs, unmounts, drops the caches and mounts again; reads the target and two other files; disarms the injector. The outcome is decided from the read alone: *intact* if the bytes read back hash to the recorded value after at least one flip, *silent corruption* if they do not and the read succeeded, *read refused* if the read failed, *no flip* if nothing was injected. A verification then remounts the volume, injector off, and hashes all twelve files. The cluster campaign runs the same attack on each node against a file of its `/data` volume. The background scrubber of beamfs ran at its default pace throughout.

Injection: `emufi 0.8.1 (srcversion 5DCD6301EB353B0FADD6FE2)` hooks the block layer with kprobes and flips one bit, at a random position, in the first segment of a read bio whose sectors match a target filter, with a probability p per hook invocation; p takes the values 10^3 , 10^5 and 10^6 parts per million. The filter is a list of sector ranges. `emufi` refuses a list of more than eight ranges or 512 bytes and then filters on the interval spanning the file: in runs 2 and 3 the per-block extent list of beamfs exceeded that limit and the interval, which includes the file’s indirect block, applied; from run 4 the bench merges contiguous extents and the exact list applied. Section VI shows that two hooks act on each beamfs read bio.

Runs: Three complete bench runs on the same image, numbered as in the project log (run 1 stopped before measuring): run 2 (bench `661e8aa`), run 3 (`60155f3`) and run 4 (`607716e`). Each writes a GPG-signed manifest of the chain it measured (the appendix). Runs 2 and 3 ended with exit status 1 because some cluster cases were not exercised: in run 2 none, the injector holding a stale range list from the filesystem comparison; in run 3 not `compute01`, whose list `emufi` refused. Run 4 ended with status 0, all twelve cluster cases exercised.

Functional tests: `xfstests [16] 2024.03.03` as packaged in the images, with three patches from the layer: beamfs added beside ext2 to the harness’s dispatch on the filesystem type, the volume kept when check declares it inconsistent, and an upstream fix to generic/589 backported. The project’s harness, `beamfs-xfstests [17]`, runs each test of the selection in its own invocation of check, on a test volume made by `mkfs.beamfs -N 16384`: the beamfs defaults, feature bits `incompat 0x7a000`, the bench’s volumes without per-inode Reed-Solomon. The selection is what check runs when given none, the `auto` group: 731 `generic` and 3 `shared` tests, 734 in all, identical on both images. One sweep per architecture on the same commit, with a budget of 14 400 s per test, aarch64 under harness version 2.3.60 and x86-64 under 2.5.1, which also records the node, its kernel, its tools, the features of the volumes it formats, its budget and every recovery of the node; generic/476 was also run alone on each architecture.

VI. AUDITING THE INJECTOR

A fault-injection result is only as good as the record of what was injected. `emufi` keeps a ring of every flip: sequence number, time, sector, operation, byte offset in the bio segment, bit, value before and after. Matching that ring against the corrections the kernel reported exposed a defect in the record, which this section explains, measures and works around. `emufi` is part of the sealed image and was not changed for this report.

A. Two hooks, one of them a bio early

`emufi 0.8.1` acts on read completions through two kprobes (Figure 2). The first, on `blk_update_request`, runs while the bio iterator is intact: it logs the bio’s start sector and flips a bit in its first segment. The block layer then calls `bio_advance` and `bio_endio`, where the second kprobe runs. `emufi` assumes that by then the iterator has been consumed, the start sector advanced past the transfer, and recovers the start by subtracting the transfer size. In Linux 7.3-rc5, `bio_advance` of a complete transfer only zeroes the size and leaves the start sector where it was (`include/linux/bio.h` at 72d3fcf8 [15]). The second hook therefore logs, and filters on, the span one bio *before* the bio it flips.

Three consequences follow. A beamfs read bio, one 4 KiB block, can take two flips, one per hook, each drawn with probability p . The second flip is logged eight sectors early. And the second hook attacks the block that follows each target block: in run 4 that put the indirect block of the target file, which follows its twelfth data block, within reach although the range list excluded it. A filesystem that reads the file in multi-block bios is touched by the first hook only, the shifted span falling before the file: ext4 here, one hook invocation per attack.

B. Recovering the hook of every flip

In ring order, an entry at sector s followed within $183\ \mu\text{s}$ by one at $s - 8$ is the pair of one bio, both flips in block

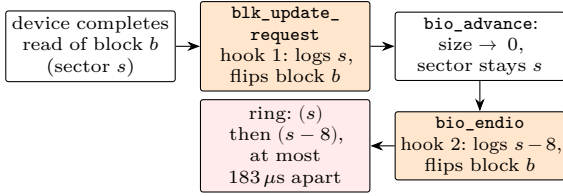


Fig. 2. The two read hooks of emufi 0.8.1 on Linux 7.3-rc5. Both flip the block that was read; the second logs it eight sectors (one 4 KiB block) early.

$s/8$. At $p = 10^6$ ppm both hooks fire whenever the filter admits them, so an entry left alone has one explanation: the first hook of a bio whose shifted span the filter refused (the first target block), or the second hook of a bio whose own block the filter refused (the block after a target block). Below 10^6 ppm either hook may have drawn alone, and a lone entry is ambiguous between its logged block and the next.

C. Checking the reconstruction

The reconstruction is checked three ways, all by the extraction script before any table is written.

- *Timing.* Over the three 10^6 ppm attacks, the two entries of a pair are at most $183 \mu s$ apart; two consecutive entries that are not a pair are at least 2.51 ms apart. No interval falls between.
- *Completeness.* At 10^6 ppm every flip is attributed (0 ambiguous).
- *The kernel's account.* beamfs logs each correction with the inode, the logical block, the codeword and the number of symbols corrected, rate-limited to the first ten lines of an attack. All 60 lines are explained by flips at the reconstructed block and codeword, or, for an ambiguous flip, at its logged block or the next; none is left over. In run 3 at 10^6 ppm one decode corrected two symbols in one codeword: the two flips of one bio, the first in that codeword's parity, the second in its data.

Table II summarises the six beamfs attacks that received flips. With one bit per hook, a decode sees at most two erroneous symbols in a codeword; the kernel reported at most 2. Taking the union over a whole attack, which overstates what any decode saw, no codeword was hit in more than 3 distinct symbols. The budget is eight. Figure 3 shows where the flips fell.

VII. RESULTS

A. Functional: *xfstests*

Table III counts the 734 tests by what check said of each, read from check's own output for every test. Both sweeps ran on commit 1bf151d, with a budget of 14 400 s per test. On x86-64 122 tests ran and all passed, generic/476, a long multi-process stress test, in 2 653 s. On aarch64 123 ran and 122 passed; the failure is generic/476, which the harness stopped at its budget after 14 388 s, before check gave a verdict. Run alone on the same node and

image with a longer budget, it passed, in 11 483 s (sweep-1791214142.results).

The other tests did not run: check declined them. On x86-64, 511 were declined because beamfs does not implement what they test, 98 because the test image or the virtual machine lacks what they need, and 3 because they apply to other filesystems only; on aarch64, 511, 97 and 3. Only generic/317 ran on one architecture and not the other: it passed on aarch64, and on x86-64 check declined it for want of procfs `uid_map` support. A declined test is untested, not passed: reflinks, quotas, `O_DIRECT`, `fallocate` modes other than allocation, encryption, extended attributes and ACLs, among others, are outside what this suite says about beamfs, and the environment's share (device-mapper targets, absent tools) is a gap of the test bed, not of beamfs.

These counts are not the ones the harness first printed. Before version 2.4.0 it recorded a declined test as passed, because check counts it in its "Passed all" line, and the aarch64 sweep was saved as 733 passes and one failure. The aarch64 counts here are recomputed from the output of check that the harness kept for each test; the x86-64 sweep was run again under 2.5.1, which records a declined test as such, its budget and every recovery of the node (Section VIII-B). Two earlier x86-64 sweeps of the same commit are superseded: one saved as 734 passes, whose per-test output was overwritten by the sweeps that followed, and one stopped after 603 tests once its budget, the default 1 900 s, had stopped generic/476.

B. Filesystem comparison

Figure 4 shows run 4 and Table IV all three runs. beamfs returned the correct file in every attack that received flips, up to 265 flips during one read, in all three runs. At 10^6 ppm, ext4 returned wrong data with a successful read after 1 flip, ext3 after 2 to 3, in every run: silent corruption. btrfs, after 2 flips, refused the read on a checksum mismatch. squashfs was never exercised: the bench builds its image offline and gave the injector no target.

The doses differ by design and are reported as received, not equalised. emufi draws once per hook and per bio, and a filesystem receives flips in proportion to the bios its read issues: one for ext4, about two per data block for beamfs (Section III-B), which is why the other filesystems received no flip at 10^3 and 10^5 ppm. What the comparison measures is the outcome per flip received: beamfs absorbed two orders of magnitude more flips than any other filesystem and returned the right bytes; ext4 and ext3 returned wrong bytes without an error; btrfs detected and refused. It is not a ranking of these filesystems in general.

After every attack the volume was remounted with the injector off and all twelve files of every filesystem hashed intact, including those of ext4 and ext3 after their silent corruption: the wrong bytes were made in the read path, not on the medium.

TABLE II

BEAMFS ATTACKS WITH FLIPS: FLIPS RECEIVED, TARGET FILTER IN EFFECT, HOOK PAIRS AND LONE ENTRIES, KERNEL LINES (ALL EXPLAINED), MOST SYMBOLS CORRECTED IN ONE DECODE (KERNEL), MOST DISTINCT SYMBOLS HIT IN ONE CODEWORD OVER THE ATTACK, COUNTING ONLY FLIPS WHOSE BLOCK IS KNOWN (NONE AT 10^5 PPM IN RUN 4, WHERE EVERY FLIP WAS ALONE).

run	ppm	flips	target	pairs	single	k-lines	dec	union
2	100 000	20	interval	2	16	10	1	1
2	1 000 000	265	interval	132	1	10	1	2
3	100 000	22	interval	1	20	10	1	1
3	1 000 000	265	interval	132	1	10	2	3
4	100 000	33	list	0	33	10	1	0
4	1 000 000	262	list	129	4	10	1	3

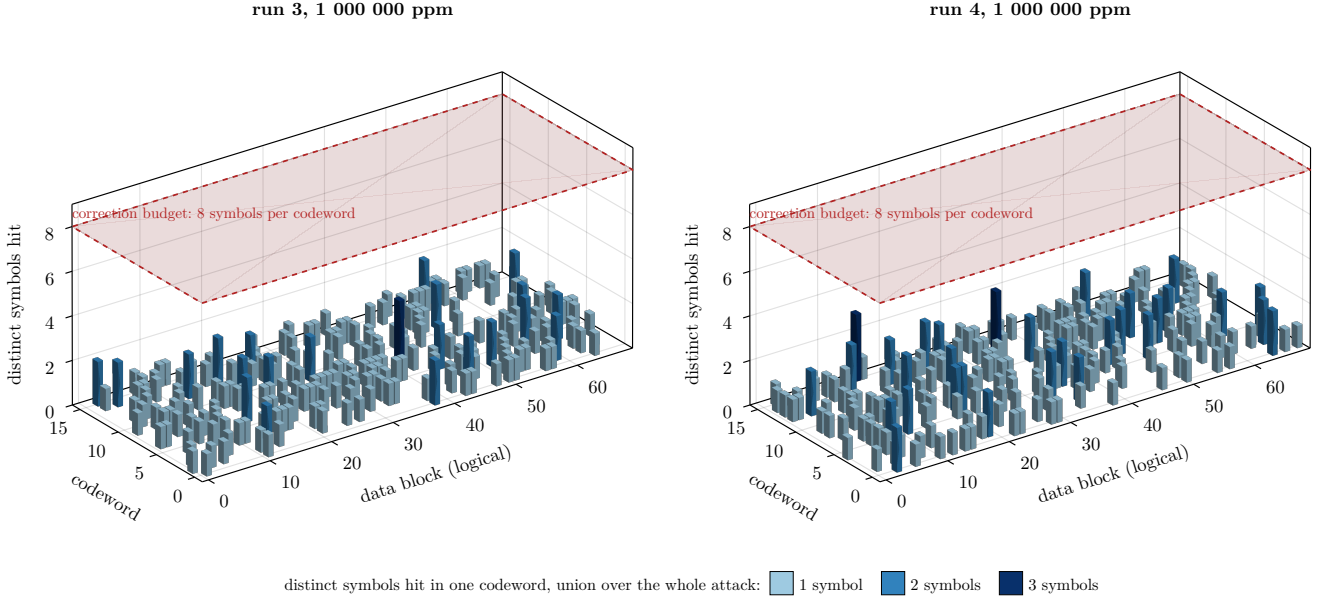


Fig. 3. beamfs at 10^6 ppm, runs 3 and 4: for each data block and codeword, the number of distinct symbols hit over the whole attack (flips of known block only). The plane marks the eight symbols one codeword corrects; no column exceeds 3.

C. Cluster

In run 4 all twelve cases were exercised (Figure 5, Table V): 9 returned the correct file after flips, 3 received none at 10^3 ppm. In run 3, `compute01` was not exercised, and `beamfs-master` at 10^6 ppm refused the read after 263 flips (Section VII-D). In run 2 no cluster case was exercised.

D. Indirect blocks

In run 3 the cluster filter was the interval spanning the file, which contains its indirect block. On `beamfs-master` at 10^6 ppm the kernel logged four times

```
beamfs/inline: corrupted indirect pointer ino=11 iblock=45
slot=33 phys=18014398509500044 (out of [17502, 262144))
```

That value is $2^{54} + 18060$: bit 54 of the pointer in slot 33 was flipped in the read buffer, the true pointer being 18060. On this code path the Reed-Solomon verify decodes a copy of the freshly read indirect block, finds a correctable error and lets the lookup proceed (Section III-C); the lookup read the pointer from the buffer, found it outside the data

area and failed. The read was refused, no wrong byte was returned, and the twelve files verified intact afterwards.

In the filesystem comparison the reconstruction of Section VI attributes 5 flips to the target file's indirect block, at slots 489 (run 2), 355 (run 2), 511 (run 3), 338 (run 3), 373 (run 4), including one in run 4 through the second hook. The file uses 57 pointers (69 data blocks, 12 of them direct), slots 0 to 56: none of these flips touched a pointer in use, and the reads were unaffected.

These events show the fail-closed contract holding for a pointer flipped out of range. They do not test the residual of Section III-C: a flip that keeps a pointer inside the data area and on an allocated block would, on these volumes, return another block's bytes.

VIII. A PARALLEL WITH PROTON IRRADIATION

Memon et al. [2], [3] exposed three commercial compute modules running Linux to 20 to 58 MeV protons at the AIC-144 cyclotron of IFJ PAN, Kraków: a Raspberry Pi Zero 2 W, an NXP i.MX 8M Plus board (Yocto Linux

TABLE III

XFSTESTS AUTO GROUP, 734 TESTS, BY WHAT CHECK SAID OF EACH TEST. DECLINED TESTS BY THE REASON CHECK GAVE; GROUPS OF FEWER THAN FOUR TESTS ARE SUMMED PER KIND, ALL 42 ARE IN THE DERIVED FILE XFSTESTS.CSV.

		x86-64	aarch64
passed		122	122
failed		0	1
declined by check		612	611
declined for			
reflink	feature	172	172
fallocate modes	feature	66	66
O_DIRECT	feature	50	50
quotas	feature	30	30
shutdown ioctl	feature	29	29
encryption	feature	26	26
extended attributes	feature	22	22
exchange-range	feature	19	19
deduplication	feature	14	14
ACLs	feature	13	13
inode flags	feature	12	12
swap files	feature	12	12
device nodes, FIFOs	feature	8	8
idmapped mounts	feature	6	6
freeze	feature	5	5
O_TMPFILE	feature	4	4
renameat2 flags	feature	4	4
13 other features	feature	19	19
device-mapper target	environment	48	48
tool absent from the image	environment	16	16
richacl	environment	9	9
device size	environment	7	7
DAX	environment	5	5
log device	environment	4	4
scsi_debug	environment	4	4
4 other needs	environment	5	4
other filesystems only	scope	3	3

5.15.77, root filesystem ext4 on eMMC) and an OrangeCrab FPGA board running a VexRiscv soft core. They record 133 Linux single-event functional interrupts across the three, and on the i.MX 8M Plus attribute 56% of failures to the filesystem and 34% to the storage driver, up to ext4 journal aborts and read-only remounts [3]. The work follows failures of unmodified Linux on commercial modules down to the kernel subsystem, across three architectures, with failure cross-sections per platform and energy. That resolution is what makes it usable by a filesystem designer, and its finding that the storage path dominates on the i.MX 8M Plus is the observation beamfs answers. Their result and ours meet at the storage path; Table VI sets the two methods side by side.

The two are complementary. A beam reproduces physics no emulator does: controller hangs, crashes, the faults their failure classes record. It is costly, and each unit absorbs dose. Emulation injects one mechanism, at chosen blocks, with a record of every flip, and leaves the hardware untouched; it cannot say what a beam does to a controller. beamfs addresses only the bytes a read returns: against a hung eMMC controller it can at best fail closed.

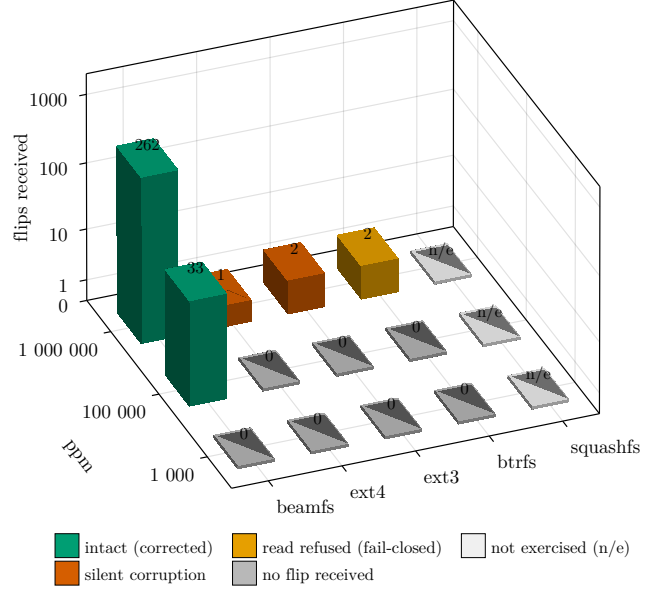


Fig. 4. Filesystem comparison, run 4: flips received during one read of the target file (logarithmic, number on each bar) and outcome, for each injection probability per hook call (ppm).

TABLE IV

FILESYSTEM COMPARISON, THREE RUNS: FLIPS RECEIVED AND OUTCOME (I INTACT, S SILENT CORRUPTION, R READ REFUSED, N NO FLIP, N/E NOT EXERCISED).

FS	ppm	run 2	run 3	run 4
beamfs	1 000	0 N	0 N	0 N
	100 000	20 I	22 I	33 I
	1 000 000	265 I	265 I	262 I
ext4	1 000	0 N	0 N	0 N
	100 000	0 N	0 N	0 N
	1 000 000	1 S	1 S	1 S
ext3	1 000	0 N	0 N	0 N
	100 000	0 N	0 N	0 N
	1 000 000	3 S	2 S	2 S
btrfs	1 000	0 N	0 N	0 N
	100 000	0 N	0 N	0 N
	1 000 000	2 R	2 R	2 R
squashfs	1 000	n/e	n/e	n/e
	100 000	n/e	n/e	n/e
	1 000 000	n/e	n/e	n/e

A. What a failure class says about the hardware

A beam acts on hardware; software only reports what follows. Telling the two apart is what lets a failure be read as a single-event effect on a sound device rather than as the symptom of a damaged one, so we read both texts on that point.

Their protocol is careful about cumulative damage. Failures are observed through software-visible channels, the kernel log, the system journal and user-space signals, logged off the device, and root causes are attributed by kernel log forensics; each run ends at its first functional interrupt [3].

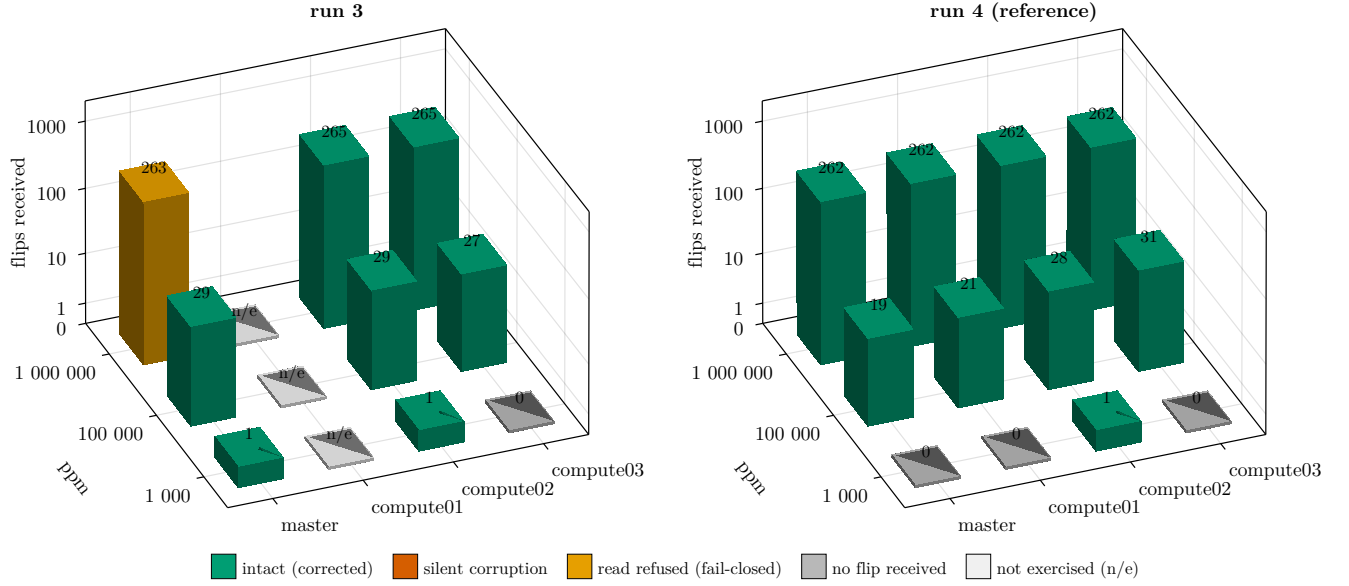


Fig. 5. Cluster campaign, runs 3 and 4: flips received by each node's attack and outcome, for each injection probability per hook call (ppm).

TABLE V
CLUSTER CAMPAIGN: FLIPS RECEIVED AND OUTCOME (CODES AS IN TABLE IV). RUN 2 EXERCISED NO CASE.

node	ppm	run 3	run 4
master	1 000	1 I	0 N
	100 000	29 I	19 I
	1 000 000	263 R	262 I
compute01	1 000	n/e	0 N
	100 000	n/e	21 I
	1 000 000	n/e	262 I
compute02	1 000	1 I	1 I
	100 000	29 I	28 I
	1 000 000	265 I	262 I
compute03	1 000	0 N	0 N
	100 000	27 I	31 I
	1 000 000	265 I	262 I

TABLE VI
PHYSICAL IRRADIATION [3] AND THIS REPORT.

	Memon et al.	this report
cause	protons, 20 to 58 MeV	emulated single-bit flips
where	SoC die (and memory, see text)	read-bio buffers of chosen blocks
effects	any: crashes, hangs, driver and filesystem errors	wrong bytes delivered to the filesystem only
filesystem	ext4 on eMMC	beamfs, ext4, ext3, btrfs, squashfs
observable	failure class per run	outcome per read; kernel corrections per decode
dose	fluence per run	flips received per attack
hardware	exposed; replaced by an unexposed unit after each fluence; not reported	not exposed; medium verified unaltered after every attack
	retested	

Supply current and voltage are logged throughout. After each targeted fluence every device is replaced by an identical, previously unexposed copy, so that total ionising dose and displacement damage do not bias the analysis [3]; the journal manuscript replaces devices after each test objective and mentions real-time detection of current latch-ups [2], while the arXiv text places single-event latch-up, micro-latch-up included, outside its scope. For one memory error the authors note that a prior clean run weakens the hypothesis of a permanent cell defect [3].

What a failure class cannot say is what the beam did to the hardware. On the i.MX 8M Plus a filesystem error has at least three physical readings, which the logs alone do not separate:

- (i) a transient upset in the system on chip, in a core, a cache, the storage host controller or a DMA engine,

that corrupts data or kernel state in flight and leaves the hardware intact;

- (ii) data altered on the eMMC itself, by an upset in the eMMC if it was in the beam, or by the system on chip writing corrupted data, which survives a power cycle;
- (iii) a part degraded by its dose, through ionising dose, displacement damage or a micro-latch-up, that no longer behaves as a fresh one.

They call for different remedies, and only the first is a property of software running on sound hardware; the second and third are effects on hardware that software then reports. The checks that separate them are a readback of the storage against its pre-exposure image, a functional and electrical test of each unit after its exposure (supply current at rest against its pre-exposure value, a memory test), and the dose each unit received. Neither text reports them, and

neither states where the units came from. Replacing units between fluences keeps damage from carrying into the next run; it does not show that a unit was intact at the end of its own run, and a cross-section computed over all the runs at one energy assumes every unit equivalent and unaltered throughout. The modules are commercial parts, not hardened ones, and their own tolerance is precisely what is left unmeasured: the campaign establishes how often these systems fail under protons, not how much the hardware itself withstands, nor, for a given failure, whether software met a transient upset or a damaged device.

The beam geometry on the i.MX 8M Plus is also stated three ways: the journal manuscript puts the system on chip and the LPDDR4 memory in the beam; the arXiv text puts only the system-on-chip die in the beam aperture, external memory and storage outside; its Table 1 lists the system on chip, the LPDDR4 and the eMMC as irradiated. Whether the eMMC and the memory were exposed changes which readings are possible for 90 % of that board’s failures.

This matters to beamfs directly. It covers reading (i) when the upset corrupts the bytes a read returns, and reading (ii) when the altered bytes stay within its correction capacity; it can do nothing for corrupted kernel state or a degraded controller, where it can at best fail closed. Which reading dominates is therefore the first question a beam campaign on beamfs must answer, and the one this emulation sidesteps by construction: here the hardware is not exposed, and the medium was read back intact after every attack (Section VII-B).

These are reporting points, not evidence against the measurements, which we take as they are reported. A beam campaign on beamfs would need them settled from the start: unit provenance and pre-exposure characterisation, dose per unit, a post-exposure functional and electrical test of every unit, a readback of the storage against its pre-exposure image, and the exact parts in the beam.

B. The same questions, asked of this report

The checks we ask of a beam campaign apply to this report too, with the instruments in the place of the hardware. Each item says what was checked and what was found.

- (a) *What was measured.* The units here are software, named by their hashes: the beamfs commit, the Yocto layer, the root image and the kernel of each architecture, and 6 compiled source trees found identical to the sources the signed manifests list (the appendix). beamfs is built into both kernels; the module file its recipe also installs is never loaded. The tools are identified by SHA-256 against the repository, not by the version they print (mkfs.beamfs prints 0.1.5, fsck.beamfs 0.1.8, the module is 0.1.26). The two kernels differ in one beamfs option: the x86-64 kernel has the tree checker `CONFIG_BEAMFS_DEBUG_TREE` (Section V); the aarch64 kernel, on which every attack ran, does not. The x86-64

xfstests sweep ran with more internal checks than the attacks.

- (b) *State after exposure.* The attacks leave the medium untouched by construction, and it was read back after every attack: twelve files intact on every filesystem (Section VII-B). For the x86-64 sweep, the node ran without a restart: the harness recorded no recovery, and the node’s uptime grew by the length of the sweep. Its kernel taint flags read 0 at the start and the same at the end, without the warning or the oops flag; no kernel warning and no oops occurred. The harness clears the kernel log before each test and keeps it after; none of the 734 logs holds a warning or an oops, and in 734 of them the tree checker reported no lost pointer.
- (c) *Dose.* Every flip is in the injector’s ring and is matched against the kernel’s own record of each correction (Section VI); doses still differ between filesystems (Section X).
- (d) *What the instruments reported.* Our own tools misreported, and each defect was found by reading a raw record against an independent account of the same event.
- emufi logs a hook one bio early on this kernel, found against the kernel’s correction lines (Section VI).
 - The xfstests harness, before 2.4.0, saved a test that xfstests declined as passed, because check counts it in its “Passed all” line; found against check’s own output for each test. Before 2.3.61 the per-test output of a sweep was overwritten by the next one. 2.4.0 asked the node for a module source version that beamfs does not expose, and recorded an empty field. All three are corrected in 2.5.0, and the x86-64 sweep was run again under 2.5.1; the counts of Table III are read from check’s output, not from the harness’s summary.
 - The first x86-64 sweep under 2.5.0 ran under the harness’s default budget of 1900s per test, below the 2317s generic/476 had taken alone on the same image, and the budget, not beamfs, stopped the test after 1883s. That budget was ours to set, and the record did not name it. The sweep was stopped after 603 tests; the harness now records its budget and every recovery of the node (2.5.1), and the sweep was run again under 14400s, in which generic/476 passed.
 - The bench’s record of each btrfs attack has no checksum line in its slice of the kernel log; the cause of the refusal, a `csum failed` warning on mirror 1, is read from the node’s own kernel log, where it appears in each run. The bench’s topology report calls beamfs a loaded module while it is built in. Correction lines are rate-limited (Section X).
- (e) *What the tools did not reach.* squashfs was never attacked: its attack printed no target in any run. The write path received no flip. xfstests declined 612 tests on x86-64 and 611 on aarch64 (Section VII-A); they

TABLE VII

GETTING CORRECT DATA BACK AFTER A FLIPPED BIT IN FILE DATA. “MEASURED” MEANS MEASURED IN THIS REPORT; OTHER ROWS ARE AS DOCUMENTED.

filesystem	on a flipped bit in file data	extra space
ext4, ext3	wrong bytes returned, no error (measured)	none
btrfs, data single	detected, read refused (measured)	checksums
btrfs, data DUP or RAID1	detected, served and repaired from the other copy [4]	a full copy
ZFS, copies=2	detected, repaired from the other copy [5]	a full copy [18]
beamfs capsule	corrected on read up to 8 symbols per codeword, measured up to 2; refused when the decoder fails; beyond 8, a wrong decode is possible without DATA_CSUM	288 bytes per 4096 (7.0 %)

are untested.

Two of these defects would have changed a result had the raw records not been read: which block each flip hit, and the xfstests counts; a third, the budget, would have reported generic/476 as failed on x86-64. The others change no number in this report; they are listed so that they are corrected before the next campaign. All are in the tools; none is in the beamfs code measured here.

IX. WHAT BEAMFS BRINGS

The measurements answer a narrow question. This section places the answer among the ways a system can get the right bytes back after a bit flips between the medium and the filesystem.

The contribution: beamfs corrects file data inside the filesystem, on one medium, for 288 bytes in 4096: 256 of parity, 16 of descriptor, 16 of generation. Redundancy also returns correct data (Table VII), at the price of a second copy of every block, and for RAID1 of a second device. The two protect against different damage: a second copy survives anything that spares it, a capsule only damage within its correction capacity. They combine, since a beamfs volume can sit on a mirror. Where a second copy does not fit, on a single small medium in an embedded, edge or space system, correction inside the filesystem is what remains; that is the case beamfs is built for. This report argues that case from the design, not from a measurement against redundancy, which is planned (Section XI).

Around that core, beamfs brings: correction on every read, not only once a checksum has failed; an error instead of bytes the decoder could not vouch for; a 64-entry correction journal in the superbblock recording block, codeword and symbols corrected; a scrubber that decodes blocks no one reads and writes corrected ones back; interleaving, so that a burst in the coded bytes is spread over sixteen codewords (Section III-A). And it comes with a measurement chain others can repeat: signed manifests, frozen snapshots, and an audit that found a defect in its own injector (Section VI).

What it costs: 288 bytes in 4096 of capacity. A Reed-Solomon encode on every write: v2 measured a median write-and-fsync latency 7.84 times that of ext4 (2.73 times at the 99th percentile), under emulation and on an earlier layout [12]; the write path has changed since and v3 measures no performance. A sequential read fetches most blocks twice (Section III-B). 17 348 lines of code outside the kernel tree, with buffer heads still under the metadata.

Where it is not better today: Pointers: in the default configuration a flipped indirect pointer that stays in range is not detected (Section III-C), where ext4 with `metadata_csum` checksums every extent block [19] and btrfs checksums data and metadata by default [20]. Inodes: one codeword each, so nine consecutive bytes lose the inode and the file with it, which interleaving cannot help (`burst-resistance.md`). Parity: grouped per codeword, so nine bytes suffice there too (Section III-A). And the project’s own notes say the default indirect-parity mode should not ship as default until indirect blocks carry a generation (Section X, item 4).

X. LIMITATIONS

Each item narrows what the results support.

- 1) *Emulation, not physics.* Every upset is a bit flipped in a read buffer by software. Nothing here is a radiation measurement, and no result transfers to a beam without one.
- 2) *The correction limit is not exercised.* At most two erroneous symbols reached a codeword in one decode, against eight correctable. Multi-bit upsets, bursts, and the regime beyond eight symbols, where a Reed-Solomon decoder can miscorrect silently (`known-limitations.md`, 3.11), are untested. The measured volumes did not carry DATA_CSUM, which exists to catch that case.
- 3) *Grouped parity.* Nine consecutive bytes in the parity of one codeword defeat it (Section III-A); burst tolerance holds for the 3 824 coded bytes only.
- 4) *Indirect blocks.* In Reed-Solomon mode the indirect parity detects and journals but does not repair the buffer; pointers 478 to 511 have no parity; a pointer flipped within the data area onto an allocated block is not detected without DATA_SELFID (Section III-C). The project’s own notes advise against Reed-Solomon as the default indirect-parity mode until a per-block generation covers indirect blocks (`known-limitations.md`, 3.13).
- 5) *The instrument.* emufi 0.8.1 logs one hook a bio early on this kernel (Section VI). Below 10^6 ppm a lone flip’s block is known only to within one block. The effective target included the block after each target block. emufi seeds its generator at load, so an attack is reproducible in its parameters, not bit for bit.
- 6) *Unequal doses.* Flips follow bios, and filesystems issue different numbers of bios for the same read. Results are reported per flip received, not at equal exposure.

- 7) *Coverage of the campaign.* Three runs; one target file per attack; static workload. squashfs was never exercised. In runs 2 and 3 part of the cluster was not exercised (Section V).
 - 8) *Functional coverage.* xfstests ran its `auto` group and declined 612 of its 734 tests on x86-64 and 611 on aarch64 (Table III); what they test is untested here. The tree checker of the x86-64 kernel was not built into the aarch64 kernel (Section V).
 - 9) *generic/476 on aarch64.* Within the sweep the harness stopped it at its time budget while it was still writing. Its reading of the volume images taken then, mounted and in use, found block pointers beyond the end of the device; an image of a volume in use cannot tell damage on disk from writes in flight, and the question is left open. Run alone on the same image the test passed, and it passed in the x86-64 sweep.
 - 10) *Tool reporting.* The bench and the xfstests harness misreported in ways listed in Section VIII-B; the results here are read from the raw records, not from their summaries.
 - 11) *Kernel log.* Correction lines are rate-limited: the per-decode figures rest on the first ten lines of each attack.
 - 12) *Platform.* Virtual machines on one host; virtual disks and USB flash media with their own controllers and error correction below the filesystem. Nothing was run on target hardware.
 - 13) *Write path.* emufi's write hook was armed but recorded no flip on beamfs; upsets on the write side are untested.
 - 14) *Size.* At 17 348 lines the module is beyond the 5 000-line audit budget the project once set itself.
- 4) *Inode replicas*, studied but not decided (`burst-resistance.md`): the one structure interleaving cannot protect.
 - 5) *Read path*, from this report: stop fetching each block twice on a sequential read, and restore on-read repair with exclusive access to the block or by copy-on-write.
 - 6) *Family B*: the reserved `SHADOW_PARITY` feature, out-of-band parity placed by stride, is the format's path toward the perturbations the scope of Section IV does not claim (`data-protection-design.md`, `format-v5.md`).
 - 7) *Security review*: stage 5 of the project roadmap, syzkaller on the module's system-call surface, afl++ on `mkfs.beamfs` and on mount-time parsing of corrupted images, a fault-injection harness, and an adversarial review against Family B (`roadmap.md`).

Measurement:

- 1) *Fix the injector* (Section VI), then inject what v3 did not: multi-bit upsets and bursts up to and beyond eight symbols per codeword, with `DATA_CSUM` on; flips on the write path; in-range pointer flips on indirect blocks.
- 2) *Equal doses*: a fixed number of flips per filesystem rather than a probability per bio.
- 3) *The right reference points*: btrfs with `DUP` data and ZFS with `copies=2`, which also return correct data, at their space cost.
- 4) *Performance* on target hardware, against ext4 and btrfs.
- 5) *The instruments' own records*, from this report: the defects of Section VIII-B corrected in the bench, the kernel's beamfs options recorded by the xfstests harness, the tests the environment declined given what they need, a budget per test set from its known duration, and the frozen aarch64 volume of generic/476 explained.
- 6) *A beam campaign* with the hardware record of Section VIII: unit provenance and pre-exposure characterisation, dose per unit, a post-exposure test of every unit, a readback of the storage against its pre-exposure image, and the exact parts in the beam.

XI. TOWARD V4

The next report has two jobs: close what this one found open, and measure what this one could not. Each item names its source, a project document at `1bf151d` or a finding of this report.

Format and code:

- 1) *Indirect blocks, whole and dated.* A parity slot of 18 codewords, 288 bytes, covering all 4 096 bytes of the block behind a new incompatible feature bit (`format-v6.md`, section 8); and a per-block generation, so that a verify can tell stale parity from damage and repair the block instead of only journalling the error (`known-limitations.md`, 3.13). Until then, the CRC mode of indirect parity refuses a block that fails its check, where the Reed-Solomon mode journals a correctable error and serves the block as read (`indparity.c`).
- 2) *DATA_CSUM and DATA_SELFID by default.* The first catches a codeword decoded wrongly beyond eight errors (`known-limitations.md`, 3.11); the second, a pointer that reaches the wrong data block (`capsule.md`). Both exist, behind `mkfs.beamfs --data-csum`, and were off on every measured volume.
- 3) *Parity interleaving*, from this report: spread the parity symbols as the data symbols are, so that nine bytes no

longer suffice anywhere in the block, and measure the cost. Reconcile the 129-byte burst the geometry gives (Section III-A) with the 139 bytes recorded from the kernel's codec (`capsule.md`).

XII. CONCLUSION

beamfs 0.1.26 passed all 122 xfstests tests that ran on x86-64 and 122 of the 123 that ran on aarch64, out of 734; xfstests declined the others, which are untested. Under emulated single-bit upsets on the read path, it never returned wrong data: it returned the correct bytes after every flip that reached a data block and refused the one read it could not vouch for. In the same conditions ext4 and ext3 returned wrong data without an error and btrfs refused. These results are bounded by their load, at most 2 erroneous symbols per codeword and per decode against eight correctable; the correction limit, bursts and miscorrection remain to be measured, with `DATA_CSUM` enabled.

Two findings matter beyond the numbers. The injector’s record was wrong in a way the outcomes alone would never have shown; checking it against the kernel’s own account of each correction found the error, and its cause in the kernel source. And the indirect-block parity, as built, detects without repairing, which leaves a pointer flipped onto another allocated block undetected on volumes without DATA_SELFID. Both are stated here so that the next measurement, on a beam or on a corrected injector, starts from them.

Is beamfs worth having? Where a second copy of the data fits, redundancy already returns correct data and beamfs is at most a complement to it. Where it does not, on one small medium, beamfs returned correct data or, once, refused, for 7% of capacity, where ext4 and ext3 returned wrong data and btrfs could only refuse (Section IX). That is a narrow claim, bounded by the load measured here, and v4 is planned to widen it or to find where it stops (Section XI).

DISCLOSURE

The analysis scripts and the draft of this report were written with the assistance of Claude Opus 5.5 (model `claude-opus-5-5`, Anthropic). Every measurement was taken by the author’s tooling on the author’s machines, and every number in the text is generated from the run records as described in [the appendix](#).

REFERENCES

- [1] R. C. Baumann, “Radiation-induced soft errors in advanced semiconductor technologies,” *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005. DOI: [10.1109/TDMR.2005.853449](#).
- [2] S. Memon *et al.*, “When Linux meets radiation: Analyzing soft errors in Linux on COTS SoCs under proton irradiation,” *IEEE Transactions on Reliability*, 2025, Published 2025-09-01; accepted manuscript at the NASA Technical Reports Server, 20250004489. Accessed: Oct. 8, 2026. [Online]. Available: <https://ntrs.nasa.gov/citations/20250004489>.
- [3] S. Memon *et al.*, *Where Linux breaks under radiation: A cross-architecture kernel-level characterization of proton-induced failures in COTS SoCs*, arXiv:2503.03722, First posted 2025-03-05 as “When Radiation Meets Linux”, 2025. Accessed: Oct. 8, 2026. [Online]. Available: <https://arxiv.org/abs/2503.03722>.
- [4] btrfs developers, *Auto-repair on read*, btrfs documentation, 2026. Accessed: Oct. 8, 2026. [Online]. Available: <https://btrfs.readthedocs.io/en/latest/Auto-repair.html>.
- [5] OpenZFS developers, *Checksums and their use in ZFS*, OpenZFS documentation, 2026. Accessed: Oct. 8, 2026. [Online]. Available: [https://openzfs.readthedocs.io/en/latest/Basic % 20Concepts /Checksums.html](https://openzfs.readthedocs.io/en/latest/Basic%20Concepts/Checksums.html).
- [6] A. Desbrières, *beamfs reference implementation, version 0.1.26*, roastercode/beamfs, commit 1bf151d9f530b44d20837db62ccdeaa7a6c7c785, 2026. [Online]. Available: <https://github.com/roastercode/beamfs>.
- [7] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960. DOI: [10.1137/0108018](#).
- [8] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam: North-Holland, 1977.
- [9] C. M. Fuchs, M. Langer, and C. Trinitis, “FTRFS: A fault-tolerant radiation-robust filesystem for space use,” in *Architecture of Computing Systems (ARCS 2015)*, ser. Lecture Notes in Computer Science, vol. 9017, Springer, 2015, pp. 96–107. DOI: [10.1007/978-3-319-16086-3_8](#).
- [10] A. Desbrières, *[RFC PATCH 0/10] FTRFS, and replies*, linux-fsdevel mailing list, Apr. 2026. [Online]. Available: <https://lore.kernel.org/linux-fsdevel/20260414120726.5713-1-aurelien@hackers.camp/T/>.
- [11] A. Desbrières, *FTRFS: Bringing radiation-robust filesystem principles to contemporary Linux*, Technical Report, Version 1, Apr. 2026. DOI: [10.5281/zenodo.19824442](#).
- [12] A. Desbrières, *BEAMFS v2: A Linux filesystem with electromagnetic resilience and Reed-Solomon recovery*, Technical Report, Version 2, Apr. 2026. DOI: [10.5281/zenodo.19886192](#).
- [13] A. Desbrières, *EMUFI v1: An electromagnetic upset fault injector for Linux filesystem resilience testing*, Technical Report, Version 1, May 2026. DOI: [10.5281/zenodo.20041762](#).
- [14] A. Desbrières, *RadFI v1, methodology paper: An algebraic fault injection operator for empirical filesystem resilience validation*, Apr. 2026. DOI: [10.5281/zenodo.19885777](#).
- [15] Linux kernel contributors, *Linux 7.3-rc5*, commit 72d3fcf802c45d00b300f25b848a93c3a2bd7c7e, torvalds/linux.git, 2026. [Online]. Available: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=72d3fcf802c45d00b300f25b848a93c3a2bd7c7e>.
- [16] fstests contributors, *Fstests: Filesystem regression test suite (xfstests)*, xfstests-dev.git, 2026. [Online]. Available: <https://git.kernel.org/pub/scm/fs/xfstests-dev.git>.
- [17] A. Desbrières, *beamfs-xfstests: An xfstests harness for beamfs on Yocto nodes*, Software, version 2.5.1, commit b22ecb4, Oct. 2026. [Online]. Available: <https://github.com/roastercode/beamfs-xfstests>.
- [18] OpenZFS developers, *Zfsprops(7): Native and user-defined properties of ZFS datasets*, OpenZFS manual, 2026. Accessed: Oct. 8, 2026. [Online]. Available:

<https://manpages.ubuntu.com/manpages/resolute/man7/zfsprops.7.html>.

- [19] Linux kernel contributors, *Ext4 data structures and algorithms: Checksums*, Linux kernel documentation, 2026. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.kernel.org/filesystems/ext4/checksums.html>.
- [20] btrfs developers, *Checksumming*, btrfs documentation, 2026. Accessed: Oct. 8, 2026. [Online]. Available: <https://btrfs.readthedocs.io/en/latest/Checksumming.html>.

APPENDIX: REPRODUCIBILITY AND CLAIMS REGISTER

Measured chain: beamfs commit 1bf151d9f530b44d20837db62ccdeaa7a6c7c785; Yocto layer 9d43172; Linux 7.3-rc5, commit 72d3fcf802c45d00b300f25b848a93c3a2bd7c7e; root images SHA-256: x86-64 af9d860bbc94127cd842c2a1bd265265c160e3628a1882ea105b20fb6210e08e, aarch64 1d60a1be893fa8eae92463082181d77be207393a3a7e18ffff7cc60183963e62; kernels SHA-256: x86-64 bzImage 837c86337113b56f4a1fc81221cdf617ec19c6bd0b53845af2a038668f6ae84f, aarch64 Image a15840c81b0d6ec74fd80145b8b8da245216e1fa055a4debe2c8cb2fd21a096a; beamfs built in on both (CONFIG_BEAMFS_FS=y), CONFIG_BEAMFS_DEBUG_TREE=y on x86-64 only; 6 compiled source trees identical to the sources the manifests list; on x86-01, mkfs.beamfs 6843a47cb41ff07c378c8db038fd1560b33169ad72adae7a61dd7fa45b4467a1 and fsck.beamfs ba772535ea050fb7e5217fcb9fe97839b0e9b27ce10628c2ab124c7870432fb8; emufi 0.8.1, srcversion 5DCD6301EB353B0FADD6FE2; bench commits 661e8aa, 60155f3, 607716e for runs 2, 3, 4.

Signed manifests: Each run manifest is signed with the author's key 319A 8EAA 89C7 538A A955 0E8B C35E E212 519E 4857 (EdDSA) and verified before extraction.

- run 2, manifest-20261005T191045Z.json: 57f2335a9fe82288307b1214b0d6975919d4f13549f2d2e4c89cf5a96e8c6e00
- run 3, manifest-20261008T064257Z.json: 506e0da692874722a70ec0e8196e6b91a63560478daf3837e5ae18340e711755
- run 4, manifest-20261008T081801Z.json: 15e9042d1e5dd57621162c31d756bfbcbcf003772ae28f34329b5f16c8236e789

xfstests records:

- x86-64 sweep, harness 2.5.1, sweep-1791491243.meta.txt 86d05bbd107cbf1f862470a0ad0e7cad54ebaf15bd0e6389848b4e3141726d76, verdicts.txt f1f450f0d5dc30e37e3702b1f682ca107b511ad6a41e2f39fb595bee9e20907c, check's output for each test, and the kernel log of each test in the harness's trace
- aarch64 sweep, harness 2.3.60, sweep-1791190821.results: 8b86f2562183fc36808a8ff500819d30240045dfaf0c2824aa6550878bd85b78, and check's output for 733 of its tests (data/raw/xfstests-evidence)
- x86-64 generic/476 alone, harness 2.3.60, sweep-1791196367.results: c0c985526f482af4b93e9374df5dbd08a79e5b4fdcb50dc1d5a62355ad5a7d39
- what the harness read in the aarch64 volumes it froze when it stopped generic/476 (data/raw/xfstests-476)
- aarch64 generic/476 alone, sweep-1791214142.results: 9130579830953f817eed7814aa55e9784613e575b3790ccb4a62586275193b37
- superseded, not counted: x86-64 sweep, harness 2.3.60, sweep-1790972412.results (dec773163aa0edc008e95cf4eb8f30a77e daa29b976b44eb3fd782b6562b9a00), saved as 734 passes, its per-test output overwritten; x86-64 sweep, harness 2.5.0, sweep-1791470226 (verdicts.txt d954c2092c1a1313b4a268eb965b55717ca1506bcea805a359ba529f55566541), stopped after 603 tests (data/raw/xfstests-superseded)

Snapshot and scripts: The whole workspace, data/raw included, is deposited with this report, together with what the measurements ran on and what ran them: the two root images and kernels above; for each architecture, the kernel configuration, the image and license manifests, the Yocto configuration with the commit of every layer, and the libvirt definition of every node with its QEMU version; and the source trees, at the commits used, of the Yocto layer, which carries beamfs, mkfs.beamfs, fsck.beamfs and emufi 0.8.1 as built, of the xfstests harness, of the bench, beamfs-bench, at the three commits it ran from, and of the Gentoo ebuilds that install the harness and the bench. The run directories, manifests, bench database and verdict files were copied once into data/raw (2532 files) and frozen by data/raw/SHA256SUMS, itself of SHA-256 6f2e4a77fea51ca737b0bdfc3b7892e730c2041ac57ab7213fa8a824a98b9667. Three scripts turn them into the paper: scripts/extract.py (516f58c0162da48b99bbb10dad79ba873d028c5916df7fe8f04b860e64ef4046) reads the raw records, cross-checks them and writes data/derived; it writes nothing if a check fails. scripts/tables.py (730219724e38b80dc7aa3ef48d5abf2973dd8b3a7541db7ce3e1e2cbf2d92f36) writes every number and table the text quotes. scripts/figures.jl (a18e643c9b20476c90e1776300b8c17f7797043d6c22c3b3ef572d35ed0effa1) draws the figures. The commands, from the workspace root:

```
cd data/raw && sha256sum -c SHA256SUMS && cd ../..
for m in data/raw/runs/manifest-*.json; do gpg --verify "$m.asc" "$m"; done
python3 -I scripts/extract.py
python3 -I scripts/tables.py
julia scripts/figures.jl
latexmk -lualatex paper.tex
```

Claims register: Each claim, the record it comes from, and the check that guards it.

claim	source	check
same beamfs commit, layer and image in runs 2 to 4	three signed manifests	GPG signature; equality across manifests (tables.py)
same beamfs sources, images and kernels on both architectures	seals, deploy directories, compiled trees (identity.txt)	image equal to seal and to the manifests; trees identical (extract.py)
xfstests: 122 of 122 run on x86-64, 122 of 123 on aarch64, declined tests apart	check's output for each test	every verdict read from it; on x86-64 equal to the harness's (extract.py)
most declined tests for features beamfs lacks	reason check gave per test	every reason classified; feature share above half (tables.py)
generic/476: passes in the x86-64 sweep; stopped by the budget in the aarch64 sweep, passes alone there	harness results and records	verdicts; seconds within the budget (extract.py , tables.py)
budget of the x86-64 sweep, 14 400 s; superseded sweep stopped within 1 900 s	meta.txt and verdicts.txt of each record	budget= lines; shell's kill report (extract.py , tables.py)
frozen aarch64 volumes of 476: pointers beyond the device	harness's reading of the images	phrase present (extract.py)
no warning or oops in the x86-64 sweep, no lost pointer	taint, uptime and recoveries in meta.txt ; kernel log of each test	taint equal, warning and oops bits clear, no restart; no such line in any log (extract.py , tables.py)
btrfs refused on a checksum mismatch	forensics-beamfs-compute01/dmesg.log of each run	csum failed lines present; none in the bench records (extract.py)
flips and outcome of every filesystem attack	multifs-all-records.txt of each run	identical to the multifs directory copy; call, flip, read status, integrity and bit counts equal to the bench database
outcome of every cluster attack	cluster-records.txt	outcome recomputed from the record equals the bench verdict
flips of an attack are the last FLIP_DELTA ring entries	flip ring in each record	ring length equals the running sum of FLIP_DELTA, every attack
two hooks per bio; pairs within 183 μ s, others ≥ 2.51 ms	flip ring	no interval between; every flip at 10^6 ppm attributed
all 60 kernel correction lines explained	kernel log slice in each record	matching on block and codeword, no line left
at most 2 symbols per decode, 3 per codeword per attack	kernel log; reconstructed flips	bound ≤ 8 (extract.py)
indirect block hit at slots 489 (run 2), 355 (run 2), 511 (run 3), 338 (run 3), 373 (run 4), none in use	reconstructed flips	slot < 57 counted (tables.py)
run 3 pointer flip, slot 33, bit 54	forensics-beamfs-master/dmesg.log of run 3	line present four times (check-paper.sh)
incompat 0x7a100 , ro_compat 0	mount line in each kernel log slice	equal in all six beamfs attacks (tables.py)
medium unaltered	verification lines	twelve files intact after every attack, all filesystems; all beamfs flips on reads
module of 17 348 lines	git show at 1bf151d	recomputed at each build (tables.py)
bio_advance leaves the sector of a complete bio	kernel source at 72d3fcf8	function body checked (check-paper.sh)
statements on Memon et al.	[2], [3], read 2026-10-08	quoted in the text